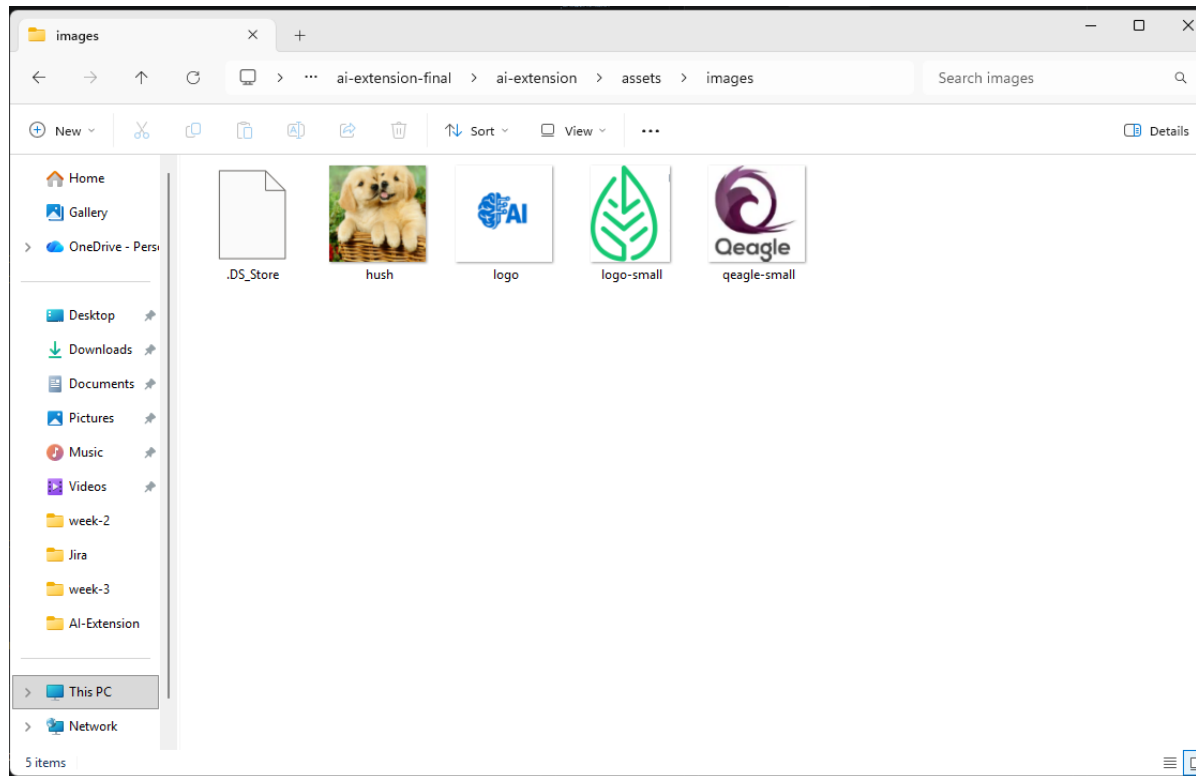


Changed the Logo of AI Extension:



EXPLORER

OPEN EDITORS

JS popup.js src/scripts

JS prompts.js src/scripts

manifest.json

chat.js src/scripts

AI-EXT...

assets/images

hush.png

logo-small.png

logo.png

qeagle-small.png

lib

src

config

appConfig.js

configUtils.js

content

content.js

scripts

api

chat.js

init-colors.js

init-config.js

popup.js

prompts.js

tabs.js

styles

bg.js

manifest.json

panel.html

manifest.json

> {} side_panel

6 "version": "1.0",

7 "description": "Test automation code generator",

8 "permissions": [

9 "storage",

10 "activeTab",

11 "scripting",

12 "tabs",

13 "sidePanel"

14],

15 "host_permissions": [

16 "call_urls"

17],

18 "action": {

19 "default_title": "Extension for AI Code Generation",

20 "default_icon": {

21 "16": "assets/images/logo.png",

22 "32": "assets/images/logo.png",

23 "48": "assets/images/logo.png",

24 "128": "assets/images/logo.png"

25 }

26 },

27 "icons": {

28 "16": "assets/images/logo.png",

29 "32": "assets/images/logo.png",

30 "48": "assets/images/logo.png",

31 "128": "assets/images/logo.png"

32 },

33 "side_panel": {

34 "default_path": "panel.html"

35 },

36 "background": {

37 "service_worker": "bg.js"

38 },

39 "web_accessible_resources": [{

40 "resources": [

41 "lib/marked/marked.min.js",

42 "lib/prism/prism.js",

43 "lib/prism/prism-typescript.js",

44 "lib/prism/prism-tomorrow.css",

45 "src/content/content.js",

46 "lib/css/all.min.css"

47]

48 }

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\Shalini\Desktop\Gen-ai\AI-Extension\ai-extension-final\ai-extension\assets\images>

CHAT

IDENTIFY IMPACTED FILES FOR PLAYWRIGHT TYPESCRIPT

Ran end.exe /c node --check src/scripts/chat.js

src/ed...

Fix applied

Restored

prompts.js

Fixed invalid template literal syntax by properly escaping all code fences inside the prompt strings

Confirmed the module now parses successfully with import("../src/scripts/prompts.js") (IMPORT_OK)

Verified src/scripts/chat.js, src/content/content.js, and src/scripts/popup.js have no syntax errors

Next step

Reload the extension in chrome://extensions

Refresh the target page

Reopen the side panel

If the error persists, send the new console output so I can verify the next failure point.fences" inside the prompt strings

Confirmed the module now parses successfully with import("../src/scripts/prompts.js") (IMPORT_OK)

Verified src/scripts/chat.js, src/content/content.js, and src/scripts/popup.js have no syntax errors

Next step

Reload the extension in chrome://extensions

Refresh the target page

Reopen the side panel

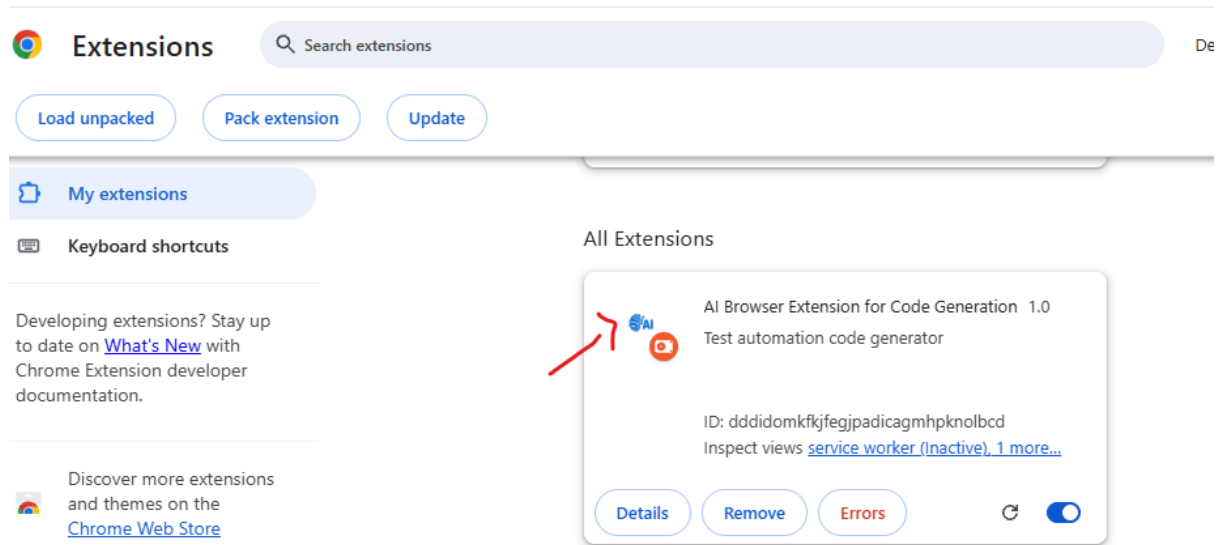
If the error persists, send the new console output so I can verify the next failure point.

manifest.json

Describe what to build

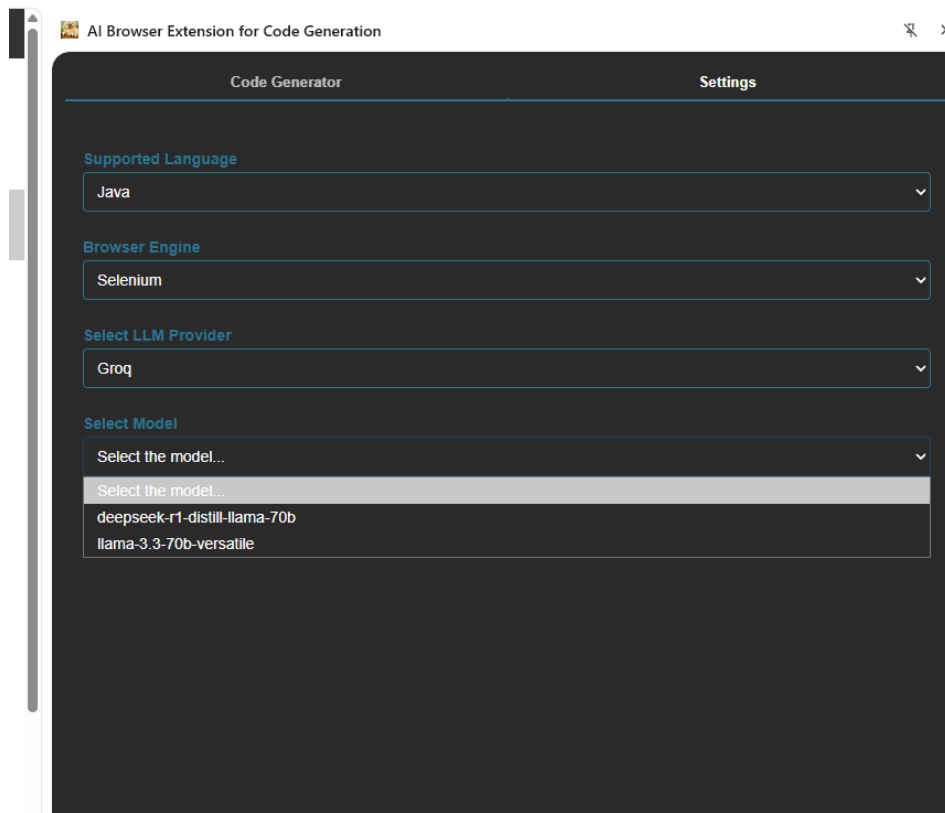
+ Auto

```
export const appConfig = {
  appName: 'AI Code Generator',
  logoPath: 'assets/images/logo.png',
  colors: {
    primary: '■ #ff6b2b',
    primaryLight: '■ #234067',
    accent: '■ rgb(43, 116, 142)',
    success: '■ #920c2e',
    successDark: '■ #a04549',
    danger: '■ #ff0000',
    darkBg: '■ #2a2a2a',
    panelBg: '■ #3a3a3a',
    mutedText: '■ #850e1a',
    // Inspector toggle specific colors
    gradientStart: '■ #275fa7',
    shadowColor: '■ rgba(255,107,43,0.3)',
    shadowColorHover: '■ rgba(255,107,43,0.4)',
    shadowColorActive: '■ rgba(255,107,43,0.1)',
    borderWhite: '■ rgba(255,255,255,0.1)'
  }
};
```



Addition of new model:

Before:



After the addition of models

Code Generator

Settings

Supported Language

Java



Browser Engine

Selenium



Select LLM Provider

Groq



Select Model

Select the model...



Select the model...

deepseek-r1-distill-llama-70b

llama-3.3-70b-versatile

OpenAI GPT-OSS-120B

OpenAI GPT-OSS-20B

Home Assignments

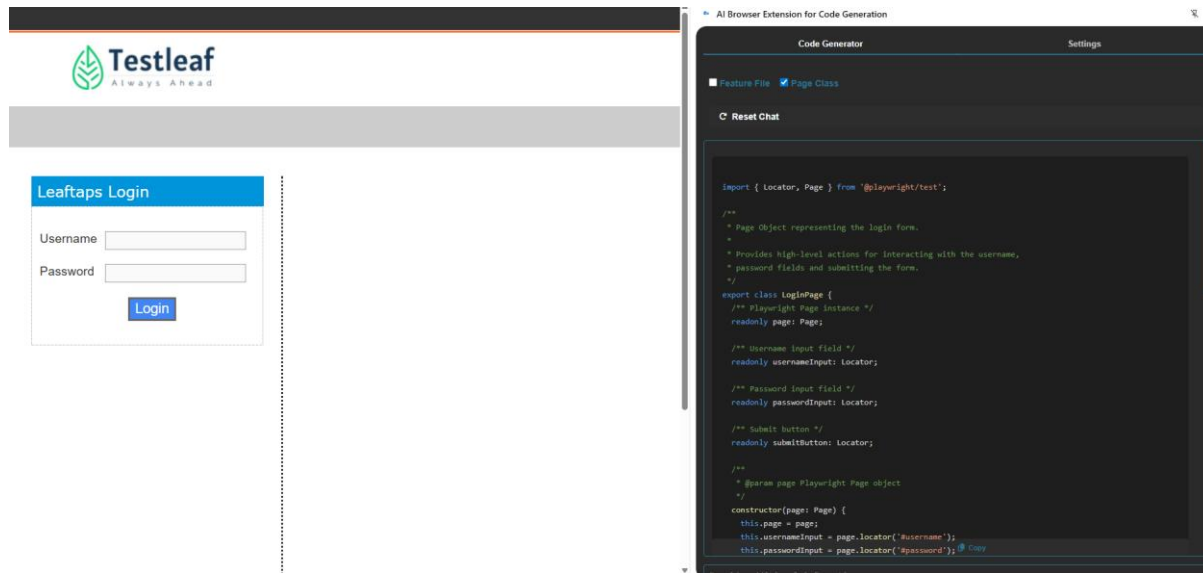
1) With vibe coding implement the playwright/typescript

The screenshot displays the Visual Studio Code interface with the following components:

- EXPLORER:** Shows the file structure of the extension, including `src/scripts`, `assets/images`, and `manifest.json`.
- MANIFEST.JSON:** The main configuration file for the extension, containing metadata like version, description, permissions, and the list of web accessible resources.
- CHAT:** A sidebar on the right showing the chat interface with a prompt: "like selenium with java playwright with typescript".
- TERMINAL:** A terminal window at the bottom showing the command prompt: `PS C:\Users\Shalini\Desktop\Gen-ai\AI-Extension\ai-extension-final\ai-extension\assets\images>`.

```
{
  "name": "ai-extension",
  "version": "1.0",
  "description": "Test automation code generator",
  "permissions": [
    "storage",
    "activeTab",
    "scripting",
    "tabs",
    "sidePanel"
  ],
  "host_permissions": [
    "<all_urls>"
  ],
  "action": {
    "default_title": "Extension for AI Code Generation",
    "default_icon": {
      "16": "assets/images/logo.png",
      "32": "assets/images/logo.png",
      "48": "assets/images/logo.png",
      "128": "assets/images/logo.png"
    }
  },
  "icons": {
    "16": "assets/images/logo.png",
    "32": "assets/images/logo.png",
    "48": "assets/images/logo.png",
    "128": "assets/images/logo.png"
  },
  "side_panel": {
    "default_path": "panel.html"
  },
  "background": {
    "service_worker": "bg.js"
  },
  "web_accessible_resources": [
    {
      "resources": [
        "lib/marked/marked.min.js",
        "lib/prism/prism.js",
        "lib/prism/prism-typescript.js",
        "lib/prism/prism-tomorrow.css",
        "src/content/content.js",
        "lib/css/all.min.css"
      ],
      "matches": "*"
    }
  ]
}
```

2)



```
import { Locator, Page, expect } from '@playwright/test';
```

```
/**
```

```
 * Login Page Object Model
```

```
 *
```

```
 * Encapsulates all login page interactions and behaviors.
```

```
 */
```

```
export class LoginPage {  
  
    // =====  
  
    // Page Instance  
  
    // =====  
  
    private readonly page: Page;  
  
  
    // =====  
  
    // Locators  
  
    // =====  
  
    readonly usernameField: Locator;  
    readonly passwordField: Locator;  
    readonly loginButton: Locator;  
  
  
    /**  
     * Constructor  
     * Initializes all page locators  
     *  
     * @param page Playwright Page instance
```

```
*/  
constructor(page: Page) {  
    this.page = page;  
  
    this.usernameField = page.locator('#username');  
  
    this.passwordField = page.locator('#password');  
  
    this.loginButton = page.locator(  
        'form#login input.decorativeSubmit[type="submit"]'  
    );  
}  
  
// =====  
// Navigation Methods  
// =====  
  
/**
```

* Navigate to Login Page

*

* @param url Application login URL

*/

```
async goto(url: string): Promise<void> {
```

```
  await this.page.goto(url, {
```

```
    waitUntil: 'domcontentloaded',
```

```
  });
```

```
}
```

```
// =====
```

```
// Action Methods
```

```
// =====
```

```
/**
```

* Enter username

*

* @param username Valid or invalid username

```
*/  
  
async enterUsername(username: string): Promise<void> {  
    await this.usernameField.fill(username);  
}
```

```
/**  
  
 * Enter password  
  
 *  
 * @param password Valid or invalid password  
 */
```

```
async enterPassword(password: string): Promise<void> {  
    await this.passwordField.fill(password);  
}
```

```
/**  
  
 * Click Login button  
 */
```

```
async clickLogin(): Promise<void> {
```

```
    await this.loginButton.click();  
}
```

```
// =====
```

```
// Business Flow Methods
```

```
// =====
```

```
/**
```

```
 * Complete Login Flow
```

```
 *
```

```
 * @param username User login name
```

```
 * @param password User password
```

```
 * @param url Optional login page URL
```

```
 */
```

```
async login(  
    username: string,  
    password: string,  
    url?: string
```

```
): Promise<void> {  
    if (url) {  
        await this.goto(url);  
    }
```

```
    await this.enterUsername(username);  
    await this.enterPassword(password);  
    await this.clickLogin();  
}
```

```
// =====
```

```
// Validation Methods
```

```
// =====
```

```
/**
```

```
 * Verify Login button is visible
```

```
 */
```

```
async verifyLoginButtonVisible(): Promise<void> {
```

```
    await expect(this.loginButton).toBeVisible();  
  }
```

```
/**
```

```
 * Verify Username field is visible
```

```
 */
```

```
async verifyUsernameFieldVisible(): Promise<void> {  
    await expect(this.usernameField).toBeVisible();  
}
```

```
/**
```

```
 * Verify Password field is visible
```

```
 */
```

```
async verifyPasswordFieldVisible(): Promise<void> {  
    await expect(this.passwordField).toBeVisible();  
}  
}
```